

Szegedi Tudományegyetem
Természettudományi és Informatikai Kar
Informatikai Intézet

SZAKDOLGOZAT

Lightweight kriptográfiai algoritmusok
teljesítmény-elemzése erőforrás-korlátozott
rendszerekre

Performance Analysis of Lightweight Cryptographic Algorithms for
Resource-Constrained Systems

Készítette:

Kárász Barna Kristóf

Mérnökinformatikus BSc hallgató

Témavezető:

Szeged
2026

Kivonat

Az Ipar 4.0 és a IoT térhódításával párhuzamosan rohamosan nő az olyan hálózatba kapcsolt eszközök száma, amelyek korlátozott processzorteljesítménnyel, szűkös memóriával és véges energiaforrással rendelkeznek. Ezek az eszközök – szenzorok, RFID tagek, beágyazott mikrovezérlők – gyakran érzékeny adatokat kezelnek, ám a hagyományos kriptográfiai megoldások futtatása rajtuk nem mindig valósítható meg hatékonyan. A jelen szakdolgozat négy kriptográfiai algoritmust vizsgál és hasonlít össze szoftveres környezetben: az AES-128-at mint ipari referenciát, a SPECK 128/128-at mint szoftver-optimalizált könnyűsúlyú megoldást, a LEA-128-at mint ISO-szabványosított blokktitkosítót, valamint az ASCON-128-at, a NIST által 2023-ban kiválasztott könnyűsúlyú kriptográfiai szabványt. Az összehasonlítás kiterjed a titkosítási és visszafejtési sebességre, a memóriaigényre, a kódméretre, a kulcsgenerálás költségére és a latenciára. A biztonsági elemzés a szakirodalom alapján foglalja össze az egyes algoritmusok ellenállóságát a differenciális, lineáris, algebrai és rotációs kriptóanalízissel, valamint az oldalsatorna-támadásokkal szemben. Az eredmények alapján alkalmazási ajánlásokat fogalmazunk meg különböző erőforrás-korlátozott környezetekre.

Kulcsszavak: lightweight kriptográfia, IoT biztonság, blokktitkosító, ASCON, SPECK, LEA, teljesítményvizsgálat

Tartalomjegyzék

1. Bevezetés	1
2. Háttérismeretek	4
2.1. Szimmetrikus kriptográfia alapjai	4
2.1.1. Blokktitkosítók	4
2.1.2. Üzemmodok	6
2.1.3. A DES és AES rövid áttekintése	7
2.2. Hash függvények és üzenethitelesítés	7
2.2.1. Kriptográfiai hash függvények	8
2.2.2. MAC – Üzenethitelesítő kód	8
2.2.3. AEAD – Hitelesített titkosítás kapcsolódó adatokkal	8
2.3. Erőforrás-korlátozott rendszerek jellemzői	9
2.3.1. Mikrovezérlő-architektúrák	9
2.3.2. Erőforrás-korlátok	10
2.3.3. Tipikus alkalmazási területek	10
2.4. A lightweight kriptográfia motivációja	11
2.4.1. Az AES korlátai korlátozott eszközökön	11
2.4.2. Az erőforrás-hatékonyság és a biztonság közötti kompromisszum	12
2.4.3. A NIST Lightweight Cryptography szabványosítási folyamat	12
3. Vizsgált algoritmusok bemutatása	13
3.1. AES-128 – referencia algoritmus	13
3.1.1. Struktúra és működési elv	13
3.1.2. Kulcsgenerálás	14
3.1.3. Az AES-128 mint referencia	14
3.2. SPECK 128/128	15
3.2.1. ARX-Feistel struktúra	15
3.2.2. Kulcsgenerálás	15
3.2.3. Szoftver-optimalizáltság	15

3.2.4.	Vitatott pontok	16
3.3.	LEA-128	16
3.3.1.	GFN struktúra – Általánosított Feistel-hálózat	16
3.3.2.	A 32 bites szóméret előnyei	17
3.3.3.	Kulcsgenerálás	17
3.4.	ASCON-128	18
3.4.1.	A szivacs-konstrukció	18
3.4.2.	Az ASCON permutáció	18
3.4.3.	Az AEAD működési folyamat	19
3.4.4.	Miért nyerte meg az ASCON a NIST versenyt?	19
3.5.	Az algoritmusok összehasonlító áttekintése	20
4.	Implementáció	21
5.	Teljesítménymérések és eredmények	22
6.	Biztonsági elemzés és összehasonlító értékelés	23
6.1.	A kriptóanalízis alapfogalmai	23
6.1.1.	Támadási modellek	23
6.1.2.	Biztonsági szint és biztonsági margó	24
6.2.	A főbb támadástípusok áttekintése	24
6.2.1.	Differenciális kriptóanalízis	24
6.2.2.	Lineáris kriptóanalízis	24
6.2.3.	Algebrai támadások	25
6.2.4.	Rotációs kriptóanalízis	25
6.2.5.	Oldalsatorna-támadások	25
6.2.6.	Születésnap-i támadás	26
6.3.	Algoritmusonkénti biztonsági profil	26
6.3.1.	AES-128	26
6.3.2.	SPECK 128/128	27
6.3.3.	LEA-128	27
6.3.4.	ASCON-128	27
6.4.	Összehasonlító biztonsági értékelés	28
7.	Összefoglalás és továbbfejlesztési lehetőségek	29
	Irodalomjegyzék	30
	Köszönetnyilvánítás	32

Nyilatkozat	33
Mellékletek	34

1. fejezet

Bevezetés

A huszonegyedik század második évtizedében az informatikai és hálózati technológiák fejlődése alapvetően formálta át az ipari termelés, a közlekedés és a mindennapi élet számos területét. Az *Ipar 4.0* néven ismertté vált paradigmaváltás keretében a gyártósorok, logisztikai láncok és épületgépészeti rendszerek egyre nagyobb mértékben támaszkodnak egymással kommunikáló, hálózatba kapcsolt eszközökre [18]. Ezzel párhuzamosan a *Dolgok Internetje* (Internet of Things, IoT) fogalma kilépett a kutatólaboratóriumok világából, és mára a hétköznapi valóság szerves részévé vált: okos otthoni termosztátok, viselhető egészségügyi eszközök, mezőgazdasági szenzorrendszerek és városi közlekedésirányító hálózatok milliárdnyi csomópontot alkotnak világszerte.

A Cisco és az Ericsson elemzése szerint 2025 végére a hálózatba kapcsolt eszközök száma meghaladja a 30 milliárdot, és ez a szám a következő évtizedben várhatóan tovább emelkedik [17]. Ebben a folyamatosan bővülő ökoszisztémában az eszközök jelentős hányada nem hagyományos számítógép vagy okostelefon, hanem korlátozott erőforrásokkal rendelkező mikrovezérlő, szenzor vagy RFID tag. Jellemzően néhány kilobájtnyi memóriával, egyetlen számjegyű megahertzben mérhető órajellel és korlátozott – gyakran akkumulátoros vagy energiagyűjtő rendszerre (energy harvesting) támaszkodó – energiaellátással dolgoznak.

E korlátozott képességű eszközök ugyanakkor gyakran rendkívül érzékeny adatokat kezelnek. Ipari környezetben a szenzorértékek és vezérlőparancsok manipulálása fizikai károkat okozhat, okos városi rendszerekben a közlekedési adatok illetéktelen módosítása balesetekhez vezethet, egészségügyi alkalmazásokban pedig a betegadatok kiszivárgása súlyos adatvédelmi következményekkel jár. Mindezek fényében a kommunikáció titkosítása és hitelesítése nem opcionális kényelmi funkció, hanem alapvető biztonsági követelmény [17].

A hagyományos kriptográfiai megoldások – elsősorban az AES-256 blokktitkosító és az RSA aszimmetrikus titkosító – bár kiváló biztonságot nyújtanak, általában asztali

számítógépek és szerverek erőforrásaira méretezettek. Egy 8 bites mikrovezérlőn, amely mindössze néhány kilobájt RAM-mal és pár tíz megahertzes órajellel rendelkezik, ezek futtatása aránytalanul sok számítási időt, memóriát és energiát igényelhet. Ez a felismerés hívta életre a *lightweight* (könnyűsúlyú) *kriptográfia* kutatási területét, amelynek deklarált célja olyan titkosítási algoritmusok és protokollok kidolgozása, amelyek minimális erőforrás-felhasználás mellett is elfogadható biztonsági szintet garantálnak [6].

A terület jelentőségét jól mutatja, hogy az Egyesült Államok Nemzeti Szabványügyi és Technológiai Intézete (National Institute of Standards and Technology, NIST) 2018-ban nyilvános pályázatot hirdetett kifejezetten könnyűsúlyú kriptográfiai szabvány kiválasztására. Az 57 beérkezett pályamunkából álló mezőnyből többkörös értékelési folyamat során, 2023 februárjában a grazi műszaki egyetem kutatócsoportja által kidolgozott ASCON algoritmust választották ki győztesnek [10]. Ez a lépés egyértelműen jelezte, hogy a lightweight kriptográfia elérte azt az érettségi szintet, amely szabványosított, széleskörűen alkalmazható megoldások születéséhez szükséges.

A jelen szakdolgozat ebbe a kontextusba illeszkedik. Célkitűzésünk négy, egymástól eltérő belső felépítésű kriptográfiai algoritmus átfogó vizsgálata és szisztematikus összehasonlítása, különös tekintettel erőforrás-korlátozott rendszerek igényeire. A vizsgált algoritmusok a következők:

1. **AES-128** – az iparban legelterjedtebb szimmetrikus blokktitkosító, amely viszonyítási alapként (referenciaként) szolgál az összehasonlításban;
2. **SPECK 128/128** – az NSA által tervezett, szoftver-optimalizált, ARX-alapú könnyűsúlyú blokktitkosító;
3. **LEA-128** – a dél-koreai NSRI által kidolgozott, ISO/IEC 29192-2:2019 szabványban rögzített könnyűsúlyú blokktitkosító;
4. **ASCON-128** – a NIST által 2023-ban szabványosított, szivacs-konstrukción alapuló hitelesített titkosítási séma.

A választás tudatos: mind a négy algoritmus eltérő belső struktúrával dolgozik (SPN, ARX-Feistel, ARX-GFN, illetve szivacs-alapú), valamennyi 128 bites kulcsot és – a blokktitkosítók esetében – 128 bites blokkot használ, továbbá mindegyik jól implementálható szoftveres környezetben. Ez az összeállítás lehetővé teszi, hogy az összehasonlítás ne csupán számszerű eredményekre, hanem a tervezési filozófiák közötti strukturális különbségek hatásaira is rávilágítson.

A dolgozat az alábbi szerkezetet követi. A 2. fejezetben a szimmetrikus kriptográfia elméleti alapjait, a hash függvények és az üzenethitelesítés fogalmait, az erőforrás-

korlátozott rendszerek jellemzőit, valamint a lightweight kriptográfia motivációját mutatjuk be. A 3. fejezet a négy kiválasztott algoritmus részletes ismertetésére és összehasonlítására tér ki. A 4. fejezetben a szoftveres implementáció módszertanát, a 5. fejezetben a teljesítménymérések eredményeit, a 6. fejezetben pedig a biztonsági elemzést és az alkalmazási ajánlásokat tárgyaljuk. Végül a 7. fejezetben összegezzük az eredményeket és felvázoljuk a lehetséges továbbfejlesztési irányokat.

2. fejezet

Háttérismeretek

A lightweight kriptográfiai algoritmusok vizsgálatához elengedhetetlen, hogy az olvasó tisztában legyen a szimmetrikus kriptográfia alapfogalmaival, a hash függvények és az üzenethitelesítés mechanizmusaival, az erőforrás-korlátozott rendszerek sajátosságaival, valamint azzal a motivációval, amely a könnyűsúlyú kriptográfia önálló kutatási területté válásához vezetett. E fejezet ezeket az alapismereteket foglalja össze, a teljesség igénye nélkül, ugyanakkor a későbbi fejezetek megértéséhez szükséges mélységben.

2.1. Szimmetrikus kriptográfia alapjai

A modern kriptográfia két nagy ága közül a *szimmetrikus kriptográfia* az, amelyben a titkosítást és a visszafejtést *ugyanaz a titkos kulcs* vezérli. Formálisan kifejezve, adott egy E titkosító és egy D visszafejtő függvény, amelyekre fennáll, hogy tetszőleges P nyílt szöveg és K kulcs esetén $D(K, E(K, P)) = P$ [1]. Az aszimmetrikus (nyilvános kulcsú) kriptográfiával szemben a szimmetrikus megoldások lényegesen kevesebb számítási erőforrást igényelnek, ami korlátozott eszközök esetében döntő szempont.

2.1.1. Blokktitkosítók

A szimmetrikus titkosítás leggyakrabban alkalmazott formája a *blokktitkosító* (block cipher), amely a bemeneti adatot fix méretű – jellemzően 64 vagy 128 bites – egységekre, úgynevezett *blokkokra* bontja, és mindegyiket a kulcs függvényében transzformálja egy azonos méretű titkosított blokkká [2]. Egy blokktitkosító belső felépítése több *körből* (round) áll, amelyek mindegyike egyszerű, önmagában gyenge műveleteket alkalmaz, de kellő számú ismétlés után az eredmény kriptográfiailag erős lesz. Claude Shannon 1949-es úttörő munkájában két alapelvelet fogalmazott meg, amelyek azóta is minden blokktitkosító tervezésének sarokkövei [3]:

- **Konfúzió** (confusion): a titkosított szöveg és a kulcs közötti statisztikai összefüggéseket a lehető legbonyolultabbá kell tenni. Ezt tipikusan nemlineáris helyettesítő műveletek (S-boxok) vagy nemlineáris aritmetikai műveletek (moduláris összeadás) biztosítják.
- **Diffúzió** (diffusion): a nyílt szöveg egyetlen bitjének megváltoztatása a titkosított szöveg minél több bitjét kell, hogy befolyásolja. Ezt permutációs rétegek, bitforgatások vagy lineáris transzformációk (pl. mátrixszorzás véges test felett) valósítják meg.

Az alábbiakban a három legfontosabb blokktitkosító-struktúrát mutatjuk be, amelyek a dolgozatban vizsgált algoritmusok mindegyikében megjelennek.

SPN – Helyettesítés-permutáció hálózat

A *Substitution-Permutation Network* (SPN) felépítésű blokktitkosító minden körben három egymást követő réteget alkalmaz. Elsőként a *helyettesítő réteg* (substitution layer) a bemeneti adatot kisebb csoportokra – általában 4 vagy 8 bites szeletekre – bontja, és mindegyiket egy előre rögzített, nemlineáris leképezésen, az úgynevezett *S-boxon* vezeti keresztül. Az S-box a konfúzió megvalósításának elsődleges eszköze: biztosítja, hogy a kimeneti bitek nemlineáris módon függjenek a bemenettől.

A második lépésben a *permutációs réteg* (permutation layer) a teljes blokk bitjeit átrendezi, ezáltal gondoskodva arról, hogy az S-boxok kimenetei a következő körben más S-boxok bemeneteire kerüljenek. Ez a diffúziós hatás teszi lehetővé, hogy néhány kör után a nyílt szöveg egyetlen bitjének megváltoztatása az egész kimeneti blokkot befolyásolja. Végül a *körkulcs hozzáadása* (round key addition) egy XOR művelettel keveri be a fő kulcsból előállított körkulcsot az aktuális állapotba. Az SPN struktúra legismertebb képviselője az AES [5].

Feistel-hálózat

A Horst Feistel nevéhez kötődő struktúrában a bemeneti blokkot két egyenlő méretű félre – L (bal) és R (jobb) – osztjuk. Minden körben az R felet egy tetszőleges F körfüggvénnyel dolgozzuk fel a körkulcs felhasználásával, az eredményt XOR-oljuk az L féllel, majd a két felet megcseréljük:

$$\begin{aligned} L_{i+1} &= R_i, \\ R_{i+1} &= L_i \oplus F(R_i, K_i). \end{aligned} \tag{2.1}$$

A Feistel-hálózat különleges előnye, hogy a visszafejtés strukturálisan megegyezik a titkosítással – csupán a körkulcsok alkalmazási sorrendjét kell megfordítani –, ami azt jelenti,

hogy az F függvénynek *nem kell invertálhatónak* lennie [2]. A DES (Data Encryption Standard) 16 körös Feistel-hálózata 1977-től évtizedeken át meghatározó szimmetrikus szabvány volt, ám 56 bites kulcsa mára nem nyújt elegendő védelmet a nyers erő (brute force) támadásokkal szemben.

ARX struktúra

Az *Add-Rotate-XOR* (ARX) megközelítés három, processzor szinten rendkívül hatékonyan végrehajtható műveletre épít [11]:

- **Add** – moduláris összeadás ($\text{mod } 2^n$): ez biztosítja a nemlinearitást (a konfúziót), anélkül, hogy memóriában tárolt S-box táblára lenne szükség;
- **Rotate** – bitenkénti forgatás (ciklikus eltolás): a diffúziót valósítja meg a szó bitjei között;
- **XOR** – kizáró vagy művelet: a körkulcs beolvasztására és további keverésre szolgál.

Az ARX struktúra elsődleges vonzereje az egyszerűség és az alacsony erőforrásigény: mindhárom művelet egyetlen gépi utasításként végrehajtható a legtöbb processzoron, nem igényel memóriában tárolt helyettesítő táblákat, és természeténél fogva *konstans idejű* (constant-time) végrehajtást tesz lehetővé, ami az oldalsatorna-támadásokkal szembeni védekezés szempontjából fontos tulajdonság. A dolgozatban vizsgált SPECK és LEA algoritmusok egyaránt ARX-alapúak, bár eltérő felépítésben alkalmazzák ezeket a műveleteket.

2.1.2. Üzem módok

Egy blokktitkosító önmagában egyetlen fix méretű adatblokkot képes titkosítani. A gyakorlatban az üzenetek hossza tetszőleges, ezért az üzenetet blokkokra kell bontani, és az egyes blokkok titkosításának módját egy *üzemmód* (mode of operation) határozza meg. A legfontosabb üzemmódok az alábbiak [1]:

- **ECB** (Electronic Codebook): Minden blokkot függetlenül, ugyanazzal a kulccsal titkosítunk. Bár egyszerű és párhuzamosítható, súlyos biztonsági hiányossága, hogy az azonos nyílt szövegű blokkok azonos titkosított blokkokat eredményeznek – a mintázatok megmaradnak a kimeneten, ami többek között képi adatokon szemléletes módon demonstrálható. Valós alkalmazásokban ezért önmagában *nem ajánlott*.
- **CBC** (Cipher Block Chaining): A titkosítás előtt minden nyílt szöveg blokkot XOR-oljuk az *előző* titkosított blokkal (az első blokknál egy véletlenszerű inicializáló vektort, IV-t használunk). Ezáltal azonos nyílt szöveg blokkok különböző titkosított

blokkokat eredményeznek. Hátránya, hogy a titkosítás nem párhuzamosítható, mivel minden blokk az előzőtől függ.

- **CTR** (Counter Mode): Egy egyedi számlálóértéket titkosítunk a blokktitkosítóval, és az eredményt (a *kulcsfolyamot*) XOR-oljuk a nyílt szöveggel. A számláló minden blokkhoz más, így tetszőleges sorrend és párhuzamosítás lehetséges. További előnye, hogy kizárólag a titkosító függvényre van szükség – a visszafejtésnél is ugyanazt alkalmazzuk.

2.1.3. A DES és AES rövid áttekintése

A *Data Encryption Standard* (DES) 1977-ben vált az Egyesült Államok szövetségi titkosítási szabványává, és hosszú évekig a legszélesebb körben alkalmazott szimmetrikus algoritmus volt. Feistel-hálózatát 16 kör alkotja, amelyekben 32 bites fél blokkokon 8 darab 6-ból-4 bites S-box végzi a nemlineáris transzformációt. A DES végzetét 56 bites kulcsmérete okozta: 1999-ben az Electronic Frontier Foundation a „Deep Crack” nevű célhardverrel mindössze 22 óra alatt végigjárta a teljes kulcsteret [2]. Bár a Triple-DES (3DES) a kulcsot háromszoros alkalmazással terjesztette ki, a megnövekedett számítási igény és a 64 bites blokkméret inherens korlátai miatt a 2000-es évekre nyilvánvalóvá vált, hogy új szabványra van szükség.

Az *Advanced Encryption Standard* (AES) 2001-ben váltotta fel a DES-t, egy nyilvános pályázati folyamat eredményeként, amelyet a NIST írt ki 1997-ben. A belga Joan Daemen és Vincent Rijmen által kidolgozott Rijndael algoritmus nyerte el a szabványosítást [4]. Az AES 128 bites blokkmérettel és 128, 192 vagy 256 bites kulcsmérettel dolgozik, SPN struktúrája 10, 12, illetve 14 körből áll. Napjainkban az AES tekinthető a szimmetrikus kriptográfia arany szabványának: hardveresen gyorsított utasításkészlet-bővítmény (Intel AES-NI) támogatja, és semmilyen ismert támadás nem fenyegeti a teljes változatot a valóságban. A jelen dolgozatban az AES-128 éppen emiatt szolgál *referenciaként* – az a kérdés, hogy a könnyűsúlyú algoritmusok milyen erőforrás-megtakarítást érnek el hozzá képest, és ezt milyen biztonsági kompromisszumok árán teszik.

2.2. Hash függvények és üzenethitelesítés

A titkosság (confidentiality) biztosítása mellett a kommunikáció biztonsága két további követelményt is magában foglal: az *integritást* (integrity), azaz annak garantálását, hogy az adatot nem módosították az átvitel során, valamint a *hitelességet* (authenticity), vagyis annak igazolását, hogy az adat valóban az elvárt feladótól származik [1]. E követelmények teljesítésében központi szerepet játszanak a kriptográfiai hash függvények, az

üzenethitelesítő kódok és – legújabb fejlődési irányként – a hitelesített titkosítási sémák.

2.2.1. Kriptográfiai hash függvények

A kriptográfiai hash függvény egy olyan H leképezés, amely tetszőleges hosszúságú bemenetet egy fix méretű kimenetre (az úgynevezett *hash értékre*, *lenyomatra* vagy *digest-re*) képez le. A biztonsági alkalmazásokhoz három alapvető tulajdonságnak kell teljesülnie [2]:

1. **Egyirányúság** (preimage resistance): adott $h = H(x)$ hash érték mellett számítási szempontból megvalósíthatatlan legyen egy olyan x' bemenet megtalálása, amelyre $H(x') = h$.
2. **Második előkép-ellenállás** (second preimage resistance): adott x bemenetnél ne lehessen hatékonyan találni egy $x' \neq x$ értéket, amelyre $H(x') = H(x)$.
3. **Ütközésmentesség** (collision resistance): ne lehessen hatékonyan találni két tetszőleges, egymástól eltérő x_1, x_2 bemenetet, amelyekre $H(x_1) = H(x_2)$.

Napjainkban a SHA-2 család (különösen a SHA-256) és a SHA-3 számítanak elfogadott kriptográfiai hash szabványoknak; a korábbi MD5 és SHA-1 algoritmusok ütközéseinek demonstrálása óta biztonsági alkalmazásokban kerülendőek.

2.2.2. MAC – Üzenethitelesítő kód

Míg egy hash függvény önmagában csak az integritást védi (bárki ki tudja számítani a hash-t, aki ismeri az üzenetet), addig az üzenethitelesítő kód (Message Authentication Code, MAC) egy *titkos kulcsot* is bevon a számításba. A MAC-ot a küldő az üzenet és a közös titkos kulcs felhasználásával állítja elő, a fogadó pedig ugyanezzel a kulccsal ellenőrzi. Illetéktelen fél – aki nem ismeri a kulcsot – nem képes érvényes MAC-ot generálni, így a sértetlenség mellett a hitelesség is biztosított. A HMAC konstrukció (Hash-based MAC) egy hash függvényre épülő, széles körben használt megoldás [1], míg a CBC-MAC a blokktitkosítók láncolási üzemmódját használja fel a hitelesítő tag előállítására.

2.2.3. AEAD – Hitelesített titkosítás kapcsolódó adatokkal

A kriptográfiai rendszerek fejlődésének egyik legfontosabb felismerése az volt, hogy a titkosítás és a hitelesítés *egyidejű*, egymással összekapcsolt biztosítása lényegesen biztonságosabb, mint a két funkció külön-külön való alkalmazása és utólagos összekapcsolása. Az *Authenticated Encryption with Associated Data* (AEAD) sémák egyetlen, integrált művelettel valósítják meg mindkettőt [9].

Egy AEAD séma négy bemeneti paraméteret vár: a titkos kulcsot (K), egy *egyszeri számot* (nonce, N), a titkosítandó nyílt szöveget (P), és a hitelesítendő, de nem titkosítandó *kapcsolódó adatot* (associated data, AD). A kimenet két részből áll: a titkosított szövegből (C) és egy *hitelesítő tagból* (T), amely az egész üzenet – beleértve a kapcsolódó adatokat is – integritását és hitelességét tanúsítja.

A kapcsolódó adatok fogalma a gyakorlati igényekből fakad: egy hálózati csomag fejléce (pl. a küldő címe, a protokolltípus) nem titkosítható, mert a hálózati eszközöknek olvasniuk kell ahhoz, hogy továbbítsák a csomagot, de *hitelesíteni* kell, mert a módosítása a csomag átirányításához vagy szolgáltatásmegtagadásos támadáshoz vezethet.

A nonce kritikus biztonsági elem: ugyanazon kulccsal *soha nem szabad kétszer* ugyanazt a nonce-ot használni, ellenkező esetben a legtöbb AEAD séma biztonsága teljes mértékben összeomlik. Ez a korlátozás a gyakorlati implementáció során kiemelt figyelmet igényel, különösen IoT környezetben, ahol az eszközök nem mindig rendelkeznek megbízható véletlenszám-generátorral.

Az AEAD paradigma központi jelentőséggel bír a jelen dolgozatban, mivel a vizsgált algoritmusok közül az ASCON-128 kifejezetten AEAD sémaként lett tervezve, ellentétben a többi vizsgált megoldással, amelyek önmagukban blokktitkosítók.

2.3. Erőforrás-korlátozott rendszerek jellemzői

A könnyűsúlyú kriptográfia létjogosultsága közvetlenül a céleszközök hardveres korlátaiból fakad. Az alábbiakban áttekintjük a legjellemzőbb mikrovezérlő-architektúrákat, azok erőforrás-korlátaikat, valamint a tipikus alkalmazási területeket, amelyekben e korlátok meghatározóvá válnak.

2.3.1. Mikrovezérlő-architektúrák

Az IoT és a beágyazott rendszerek világában a processzorokat szóhosszuk alapján három fő kategóriába sorolhatjuk. A *8 bites mikrovezérlők* – mint például az Atmel ATmega család, amely az Arduino platformok alapja – jellemzően 1–20 MHz órajellel, 1–8 KB RAM-mal és legfeljebb néhány száz kilobájt flash memóriával rendelkeznek. Áruk néhány tíz centtől néhány dollárig terjed, ami tömeges alkalmazásokban – RFID tagek, egyszerű szenzorcsomópontok – döntő szempont.

A *16 bites mikrovezérlők* (pl. Texas Instruments MSP430 család) a vezeték nélküli szenzor-hálózatok (Wireless Sensor Network, WSN) leggyakrabban alkalmazott platformjai. Tipikusan 1–25 MHz órajelet, 2–16 KB RAM-ot és 16–256 KB flash memóriát kínálnak, miközben kifejezetten alacsony energiafogyasztásra optimalizáltak – alvó üzemmódban akár néhány mikroampernyi áramfelvétellel is képesek működni [6].

A *32 bites mikrovezérlők* – elsősorban az ARM Cortex-M család tagjai (M0, M3, M4) és az ESP32 – képviselik az IoT eszközök csúcskategóriáját. Órajelük 16–240 MHz között mozog, 16–512 KB RAM áll rendelkezésükre, és flash kapacitásuk meghaladhatja az 1 MB-ot. Ipari IoT (IIoT) gateway-ek, okos otthoni központok és komplex szenzorrendszerek építőelemeiként szolgálnak. Bár erőforrásaik a fenti kategóriák közül a legbőségesebbek, hagyományos asztali vagy szerver processzorokhoz képest még mindig nagyságrendi különbségek mutatkoznak mind a számítási kapacitás, mind a rendelkezésre álló memória tekintetében.

2.3.2. Erőforrás-korlátok

A korlátozott eszközök három fő erőforrás-dimenziója a *memória*, a *számítási kapacitás* és az *energia*.

A memória két összetevője az illékony RAM (volatile), amely a futásidejű változókat, köztes értékeket és kulcsokat tárolja, valamint a nem illékony flash (non-volatile, ROM jellegű), ahol a programkód és a konstans adatok – például S-box táblák – foglalnak helyet. Egy AES-128 implementáció jellemzően 200–400 bájt RAM-ot igényel az előre kiszámított körkulcsok és az S-box tábla tárolásához, ami 8 bites platformokon, ahol a teljes RAM néhány kilobájtra korlátozódik, az elérhető memória jelentős hányadát foglalja el [8]. RFID tageknél a helyzet még drámaibb: az alkalmazás számára elérhető RAM akár 300–500 bájtra is csökkenhet.

A számítási kapacitás korlátja abban nyilvánul meg, hogy az alacsony órajel mellett gyakran hiányzik a hardveres szorzó vagy a lebegőpontos egység, és minden kriptográfiai művelet szoftveres úton, az alapvető ALU utasítások felhasználásával hajtható csak végre. A teljesítményt ilyen környezetben a *ciklus/bájt* (cycles per byte) mérőszámmal szokás jellemezni, ami hardver- és implementációfüggetlen összehasonlítást tesz lehetővé.

Az energiakorlátok különösen a telepről működő vagy energiagyűjtő (energy harvesting) eszközöknél válnak kritikussá. Minden végrehajtott óraciklus energiát fogyaszt, így a kriptográfiai műveletek közvetlen hatással vannak az eszköz élettartamára. Egy olyan szenzor esetében, amelynek évekig kell működnie egyetlen gombelem energiájával, a titkosításra fordított energiamennyiség nem elhanyagolható tervezési szempont [6].

2.3.3. Tipikus alkalmazási területek

Az erőforrás-korlátozott kriptográfia iránti igényt számos, egymástól markánsan különböző alkalmazási terület generálja. Az *RFID rendszerekben* (útleveél chippek, raktári címkék, beléptető kártyák) elsősorban a hitelesítés és az integritásvédelem a cél, minimális hardveren. A *vezeték nélküli szenzor-hálózatokban* (WSN) a szenzor-csomópontok közötti

kommunikáció titkossága és integritása szükséges, miközben az eszközök akkumulátoros működése határt szab a számítási kapacitásnak.

Az *ipari IoT* (Industrial IoT, IIoT) területén PLC vezérlők, ipari szenzorok és SCADA rendszerek kommunikációjának biztonságát kell garantálni, ahol a tét különösen magas: egy kompromittált ipari vezérlőrendszer fizikai károkat is okozhat [18]. Az *okos város* alkalmazásokban közlekedési szenzorok, intelligens közvilágítási rendszerek és parkolás-figyelő csomópontok adatainak védelméről van szó, míg az *egészségügyi eszközöknél* az implantátumok és viselhető fitness-monitorok által kezelt személyes egészségügyi adatok védelme a GDPR szabályozás tükrében különösen szigorú követelményeket támaszt.

2.4. A lightweight kriptográfia motivációja

Miután áttekintettük a szimmetrikus kriptográfia alapjait és a korlátozott eszközök jellemzőit, természetes módon adódik a kérdés: ha az AES-128 a jelenlegi arany szabvány, miért van szükség egyáltalán más megoldásokra? A válasz nem egyszerű igennel vagy nemmel adható meg – inkább egy kompromisszumok által meghatározott spektrumról van szó.

2.4.1. Az AES korlátai korlátozott eszközökön

Az AES-128 önmagában kiváló algoritmus, amely a szimmetrikus kriptográfia több évtizedes tapasztalatán nyugszik. Ugyanakkor tervezésekor elsősorban asztali és szerver környezetre optimalizáltak [5]. A 8 bites S-box tábla 256 bájt memóriát foglal el – ami egy 2 KB RAM-mal rendelkező mikrovezérlőn az összes memória nyolcadát jelenti. A MixColumns lépés a $GF(2^8)$ véges test feletti mátrixszorzást igényel, ami 8 bites processzorokon különösen számításigényes. Hardveres megvalósításban az AES mintegy 2400 kapu-ekvivalens (Gate Equivalent, GE) területet foglal el, ami ultra-kis chipen – például passzív RFID tageken, ahol a teljes áramkör területe néhány ezer GE-re korlátozódik – nem engedhető meg [6].

Mindez nem jelenti azt, hogy az AES minden korlátozott eszközön használhatatlan – a 32 bites ARM Cortex-M processzorok például kényelmesen futtatják –, de az erőforrás-spektrum alsó felén komoly hatékonysági problémák merülnek fel. A könnyűsúlyú algoritmusok célja, hogy pontosan ezt a rést töltsék ki: kisebb S-boxokkal (vagy S-box nélkül), egyszerűbb műveleti struktúrával és kevesebb memóriaigénnyel dolgozzanak, miközben a biztonsági szint az adott alkalmazáshoz elegendő marad.

2.4.2. Az erőforrás-hatékonyság és a biztonság közötti kompromisszum

A kriptográfiában alapvető összefüggés, hogy a biztonság növelése általában több erőforrást igényel. Több titkosítási kör jobb biztonságot nyújt, de több számítást és energiát fogyaszt; nagyobb S-box erősebb nemlinearitást biztosít, de több memóriát foglal; hosszabb kulcs szélesebb kulcsteret jelent, de nagyobb tárolási és feldolgozási igényel jár.

A könnyűsúlyú kriptográfia lényege éppen e kompromisszum tudatos kezelése. Egy IoT szenzornak, amelynek élettartama néhány év, és amelyet zárt, ellenőrzött hálózatban használnak, nem feltétlenül van szüksége 256 bites biztonsági szintre – a 128 bites biztonság, amely mintegy $3,4 \times 10^{38}$ műveletet igényel a feltöréshez, a belátható jövőben is elegendő védelmet nyújt [7]. A tervezési kihívás tehát abban rejlik, hogy megtaláljuk azt a pontot, ahol a biztonsági szint a konkrét alkalmazás számára még elfogadható, miközben az erőforrás-felhasználás a lehető legalacsonyabb.

2.4.3. A NIST Lightweight Cryptography szabványosítási folyamat

A könnyűsúlyú kriptográfia területének mérföldkövét jelentette, amikor a NIST 2018-ban nyílt pályázatot hirdetett kifejezetten IoT és más korlátozott környezetekre szánt kriptográfiai szabvány kiválasztására. A pályázat konkrét követelményeket fogalmazott meg: a jelölteknek AEAD funkcionalitást (és opcionálisan hash funkciót) kellett biztosítaniuk legalább 128 bites biztonsági szinttel, miközben teljesítményük korlátozott eszközökön szignifikánsan jobb kellett legyen a meglévő NIST szabványoknál [10].

Az első fordulóban 57 pályázat érkezett a világ kriptográfiai kutatóközösségéből. A NIST több éven átívelő, transzparens értékelési folyamatot folytatott, amelynek során a jelölteket biztonsági elemzéseknek, teljesítményméréseknek és harmadik féltől származó kriptóanalízisnek vetették alá. 2021-re a mezőny 10 döntőre szűkült, majd 2023 februárjában a NIST az *ASCON* algoritmust választotta ki győztesként.

Az *ASCON* sikere több tényezőnek volt köszönhető: elegáns és egyszerű belső felépítése könnyen implementálhatóvá tette mind szoftverben, mind hardverben; 320 bites belső állapota kellően kompakt; egyetlen permutációra épülő konstrukciója egyszerre biztosított AEAD és hash funkcionalitást; és az éveken át tartó nyilvános kriptóanalízis nem tárt fel hatékony támadási lehetőséget [9]. A szabványosítási folyamat maga is fontos tanulságokkal szolgál: rámutatott arra, hogy a könnyűsúlyú kriptográfia mára kellően érett terület ahhoz, hogy ipari szabványok alapjául szolgáljon, és hogy a nyílt, versenyen alapuló kiválasztási modell – amelyet korábban az AES esetében is alkalmaztak – itt is eredményesen működik.

3. fejezet

Vizsgált algoritmusok bemutatása

E fejezetben részletesen bemutatjuk a dolgozat keretében vizsgált négy kriptográfiai algoritmust. A kiválasztás szempontjai a struktúra-diverzitás, az egységes 128 bites kulcsméret és – ahol értelmezhető – a 128 bites blokkméret, a szabványosítottsági szint, valamint a szoftveres implementálhatóság voltak. Az algoritmusok bemutatása kiterjed azok történeti hátterére, belső működésére, kulcsgenerálási mechanizmusára és a korlátozott eszközökön való alkalmazhatóság szempontjából releváns tulajdonságaira.

3.1. AES-128 – referencia algoritmus

Az *Advanced Encryption Standard* (AES) a szimmetrikus kriptográfia legszélesebb körben elfogadott és alkalmazott blokktitkosítója. Eredeti neve *Rijndael* – a belga Joan Daemen és Vincent Rijmen nevéből képzett szóösszevonás –, és a NIST által 1997-ben kiírt nyilvános pályázat győzteseként 2001-ben nyerte el az AES szabvány (FIPS 197) státuszt [4]. A pályázaton 15 jelölt indult; a Rijndael algoritmust biztonsági erőssége, hatékonysága és implementációs rugalmassága emelte a mezőny fölé [5].

3.1.1. Struktúra és működési elv

Az AES-128 egy SPN (Substitution-Permutation Network) struktúrájú blokktitkosító, amely 128 bites blokkokat dolgoz fel 128 bites kulccsal, 10 körön keresztül. A belső állapotot egy 4×4 -es bájtos mátrix (az ún. *State*) reprezentálja. Minden kör négy, jól meghatározott transzformációt alkalmaz:

1. **SubBytes** – Minden bájt egy 256 elemű S-box táblázaton halad keresztül, amelynek matematikai alapja a $GF(2^8)$ véges test feletti multiplikatív inverz, kiegészítve egy affin transzformációval. Ez a lépés biztosítja a konfúziót: a kimeneti bitek nem-lineáris módon függenek a bemenettől.

2. **ShiftRows** – A State mátrix sorait ciklikusan balra tolja: a nulladik sort 0, az első 1, a másodikat 2, a harmadikat 3 pozícióval. Ez a diffúzió első eleme, amely a bájtokat különböző oszlopokba keveri.
3. **MixColumns** – Minden oszlopot egy fix mátrixszal szoroz meg a $GF(2^8)$ véges test felett. Ez a lépés az oszlopokon belüli diffúziót valósítja meg, biztosítva, hogy egyetlen bemeneti bájt megváltoztatása az oszlop mind a négy kimeneti bájtját befolyásolja. Az utolsó (10.) körben ez a lépés kimarad.
4. **AddRoundKey** – A State mátrixot bitenként XOR-olja az aktuális körkulccsal, amely a fő kulcsból a kulcsgenerálás (key schedule) során áll elő.

3.1.2. Kulcsgenerálás

Az AES-128 kulcsgenerálása a 128 bites fő kulcsból 10 darab 128 bites körkulcsot állít elő (plusz az eredeti kulcs maga is körkulcsként szolgál a nulladik körben, így összesen 11 darab 128 bites kulcsblokk keletkezik). A folyamat oszlopszintű műveletekre épül: az előző körkulcs utolsó oszlopát ciklikusan eltoljuk, bájtjait az S-boxon vezetjük át, majd egy körkonstanssal (round constant, Rcon) XOR-oljuk, végül az eredményt az előző körkulcs első oszlopával is XOR-olva kapjuk az új körkulcs első oszlopát. A többi oszlop az előző oszlop és a megfelelő korábbi oszlop XOR-jaként áll elő [5].

3.1.3. Az AES-128 mint referencia

A dolgozatban az AES-128 nem könnyűsúlyú algoritmusként, hanem *viszonyítási alapként* szerepel. Mivel a világ informatikai infrastruktúrájának döntő része erre a titkosítóra épül – banki rendszerek, VPN kapcsolatok, TLS protokollok egyaránt alkalmazzák –, a lightweight algoritmusok értékelésénél az a központi kérdés, hogy milyen erőforrás-megtakarítást érnek el az AES-hez képest, és ezt milyen áron teszik [14].

Ugyanakkor az AES-128-nak korlátozott eszközökön érzékelhető hátrányai is vannak. A 256 bájtos S-box tábla memóriát foglal, a MixColumns lépés véges test feletti aritmetikája 8 bites processzorokon számításigényes, és a hardveres implementáció mintegy 2400 GE-t igényel. Modern asztali processzorokban az Intel AES-NI utasításkészlet-bővítmény hardveresen gyorsítja az AES-t, de mikrovezérlőkön ilyen támogatás általában nem áll rendelkezésre.

3.2. SPECK 128/128

A *SPECK* algoritmust az Egyesült Államok Nemzetbiztonsági Ügynöksége (NSA) fejlesztette ki és publikálta 2013-ban, a testvér-algoritmus SIMON-nal együtt [11]. Míg a SIMON elsősorban hardveres implementációkra optimalizált, a SPECK deklaráltan *szoftveres teljesítményre* lett tervezve. A család több változatot tartalmaz eltérő blokk- és kulcsméretekkel; a jelen dolgozatban a *SPECK 128/128* változatot vizsgáljuk, amely 128 bites blokkot és 128 bites kulcsot használ.

3.2.1. ARX-Feistel struktúra

A SPECK 128/128 belső állapota két 64 bites szóra (x, y) oszlik. A titkosítás 32 körből áll, és minden kör az alábbi ARX-alapú Feistel-jellegű transzformációt hajtja végre:

$$x \leftarrow ((x \ggg 8) \boxplus y) \oplus K_i, \quad (3.1)$$

$$y \leftarrow (y \lll 3) \oplus x, \quad (3.2)$$

ahol $\ggg$ és $\lll$ a jobbra, illetve balra forgatást, $\boxplus$ a moduláris összeadást ($\text{mod } 2^{64}$), $\oplus$ az XOR műveletet, K_i pedig az i -edik körkulcsot jelöli. A struktúra rendkívüli egyszerűsége szembetűnő: egyetlen kör mindössze egy forgatást, egy összeadást, egy XOR-t és még egy forgatást tartalmaz. A nemlinearitást kizárólag a moduláris összeadás biztosítja a carry propagation (átvitel-terjedés) révén [11].

3.2.2. Kulcsgenerálás

A SPECK 128/128 kulcsgenerálása figyelemre méltó módon *ugyanazt az ARX struktúrát* használja, mint maga a titkosítás. A 128 bites fő kulcs két 64 bites szóra oszlik: $K = (l_0, k_0)$, ahol k_0 az első körkulcs. A további körkulcsokat az alábbi rekurzióval generáljuk:

$$l_{i+1} = (k_i \boxplus (l_i \ggg 8)) \oplus i, \quad (3.3)$$

$$k_{i+1} = (k_i \lll 3) \oplus l_{i+1}. \quad (3.4)$$

Ez a megoldás két lényeges előnnyel jár: egyrészt a kulcsgenerálás nem igényel az alap-titkosítástól különböző műveleteket (ami a kód méretet minimalizálja), másrészt lehetővé teszi az *on-the-fly* körkulcs-előállítás, vagyis a körkulcsok menet közben generálhatók anélkül, hogy az összeset előre el kellene tárolni a RAM-ban.

3.2.3. Szoftver-optimalizáltság

A SPECK szoftveres hatékonyságának kulcsa az ARX műveletek processzor-szintű támogatottságában rejlik. A moduláris összeadás egyetlen ADD utasítás, a forgatás egyet-

len ROR/ROL utasítás, az XOR egyetlen XOR utasítás a legtöbb ISA-ban (utasításkészlet-architektúrában). Nincs szükség memóriában tárolt S-box táblára, ami egyrészt memóriát takarít meg, másrészt kiküszöböli a cache-elérési mintákból fakadó oldalsatorna-szivárgás kockázatát.

A teljes SPECK 128/128 titkosítás C nyelven kevesebb mint 10 sor kódból implementálható, ami a kódméret szempontjából rendkívül kedvező. Dinu és munkatársai a „Triathlon of Lightweight Block Ciphers” benchmarkjában kimutatták, hogy a SPECK az egyik leggyorsabb szoftveres könnyűsúlyú titkosító mikrovezérlő-platformokon [14].

3.2.4. Vitatott pontok

A SPECK körül kialakult viták elsősorban nem biztonsági, hanem *bizalmi* jellegűek. Az NSA nem publikálta a tervezési döntések teljes indoklását (design rationale) – nem tette közzé, miért éppen a 8 és 3 bites forgatási paramétereket választotta, és miért 32 kört alkalmazott –, ami a kriptográfiai közösség egy részében gyanakvást keltett. Az ISO/IEC szabványosítási kísérletet nem biztonsági aggályok, hanem a tervezési transzparencia hiánya miatt utasították el. Fontos hangsúlyozni ugyanakkor, hogy a SPECK ellen évtizedes független kriptóanalízis folyt, és a teljes változat ellen *nem találtak a brute force-nál hatékonyabb támadást* [11].

3.3. LEA-128

A *Lightweight Encryption Algorithm* (LEA) a dél-koreai Nemzeti Biztonsági Kutatóintézet (NSRI – National Security Research Institute) fejlesztése, amelyet Hong és munkatársai publikáltak 2013-ban a WISA konferencián [12]. A LEA-t a kezdetektől fogva *közönséges processzorokon való gyors szoftveres futásra* tervezték, különös tekintettel a 32 bites mikrovezérlőkre és beágyazott platformokra. Az algoritmus nemzetközi elismerését jelzi, hogy 2019-ben bekerült az ISO/IEC 29192-2 szabványba mint könnyűsúlyú blokktitkosító [13], emellett a koreai KCMVP (Korean Cryptographic Module Validation Program) nemzeti szabványként tartja nyilván.

3.3.1. GFN struktúra – Általánosított Feistel-hálózat

A LEA-128 belső felépítése az ARX műveleti készletet egy *Generalized Feistel Network* (GFN, Általánosított Feistel-hálózat) szerkezetben alkalmazza. Míg a hagyományos Feistel-hálózat két félre, a SPECK-féle megoldás két 64 bites szóra osztja a blokkot, addig a LEA négy darab 32 bites szóra bontja a 128 bites bemenetet: (X_0, X_1, X_2, X_3) . Minden egyes

kör az alábbi transzformációkat hajtja végre [12]:

$$X'_0 = \text{ROL}((X_0 \oplus RK_0) \boxplus (X_1 \oplus RK_1), 9), \quad (3.5)$$

$$X'_1 = \text{ROR}((X_1 \oplus RK_2) \boxplus (X_2 \oplus RK_3), 5), \quad (3.6)$$

$$X'_2 = \text{ROR}((X_2 \oplus RK_4) \boxplus (X_3 \oplus RK_5), 3), \quad (3.7)$$

$$X'_3 = X_0, \quad (3.8)$$

ahol $RK_0, \dots, RK_5$ az adott kör hat darab 32 bites körkulcsa, $\boxplus$ a moduláris összeadás ($\text{mod } 2^{32}$), és ROL/ROR a balra/jobbra forgatás. A LEA-128 változat 24 kört alkalmaz.

Érdemes kiemelni a struktúra két jellegzetes vonását. Egyrészt a négyágú Feistel-hálózat gyorsabb diffúziót biztosít, mint a kétágú: a négy szó közötti keverés kevesebb körben éri el a teljes avalanche hatást. Másrészt a körönkénti hat körkulcsszó (összesen $24 \times 6 = 144$ darab 32 bites kulcsszó) rendkívül intenzív kulcsbevonást jelent, ami megnéhezíti a kulccsal kapcsolatos támadásokat.

3.3.2. A 32 bites szóméret előnyei

A LEA és a SPECK között talán a legszembevetőbb különbség a szóméretben rejlik. A SPECK 128/128 két 64 bites szóra bontja a blokkot, míg a LEA négy 32 bites szóval dolgozik. Ez a különbség a 32 bites mikrovezérlőkön – amelyek az IoT eszközök legnépesebb családját alkotják – komoly gyakorlati következményekkel jár: a SPECK 64 bites moduláris összeadása egy 32 bites processzoron *két utasítást* igényel (alsó és felső 32 bit kezelése az átvitel propagálásával), míg a LEA 32 bites összeadása *egyetlen natív utasítás-ként* hajtható végre. Ennek köszönhetően a LEA 32 bites ARM Cortex-M processzorokon gyakran *gyorsabb*, mint a SPECK [12].

3.3.3. Kulcsgenerálás

A LEA-128 kulcsgenerálása négy darab 32 bites konstansra ($\delta_0, \delta_1, \delta_2, \delta_3$) épül, amelyek értéke a tervezők által nyilvánosan indokolt hexadecimális állandó. A 128 bites fő kulcsot négy 32 bites szóra (K_0, K_1, K_2, K_3) bontjuk, és az i -edik kör kulcsszavait az alábbi rekurzió generálja:

$$T_j = \text{ROL}(K_j \boxplus \text{ROL}(\delta_{i \bmod 4}, i + j), r_j), \quad j = 0, 1, 2, 3 \quad (3.9)$$

ahol $r_0 = 1, r_1 = 3, r_2 = 6, r_3 = 11$ a forgatási paraméterek, és a hat körkulcsszóból néhány a T_1 értékét ismétli ($RK = (T_0, T_1, T_2, T_1, T_3, T_1)$), végül a fő kulcsszavakat frissítjük: $K_j \leftarrow T_j$. A kulcsgenerálás tehát – hasonlóan a SPECK-hez – ARX műveletekre épül, és on-the-fly is végrehajtható [12].

3.4. ASCON-128

Az *ASCON* algoritmust a grazi műszaki egyetem (TU Graz, Ausztria) kutatócsoportja fejlesztette ki: Christoph Dobraunig, Maria Eichlseder, Florian Mendel és Martin Schl  ffer 2014-ben ny  jtott  k be a CAESAR (Competition for Authenticated Encryption: Security, Applicability, and Robustness) versenyre, majd az algoritmus a NIST 2018-ban ind  tott Lightweight Cryptography szab  nyos  t  si folyamat  nak gy  ztesek  nt 2023-ban nemzetk  zi szab  v  nny   v  lt [10]. Az ASCON paradigmatikusan elt  r a kor  bban bemutatott h  rom algoritmust  l: nem hagyom  nyos blokktitkos  t  , hanem egy *szivacs-konstruk  ci  n* (sponge construction) alapul  , nat  van AEAD   s hash funkcionalit  st biztos  t   kriptogr  fiai primit  v.

3.4.1. A szivacs-konstruk  ci  

A szivacs-konstruk  ci   egy permut  ci  -alap   kriptogr  fiai keretrendszer, amelyet eredetileg a SHA-3 (Keccak) hash f  ggv  nyn  l alkalmaztak. L  nyege, hogy egy nagy m  ret   bels     llapotot (state) k  t r  szre oszt  : a *rate* (r)   s a *capacity* (c) r  szre. A rate r  sz a nyilvánosan el  rhet   interf  sz – ezen kereszt  l t  rt  nik az adatok beolvas  sa (absorption, elnyel  s)   s a kimenet el    ll  t  sa (squeezing, k  pr  sel  s) –, m  g a capacity r  sz k  v  lr  l soha nem   rhet   el k  zvetlen  l,   s   ppen ez az elk  l  n  t  s biztos  tja a kriptogr  fiai biztons  got [9].

Az ASCON-128 320 bites bels     llapota   t darab 64 bites sz  b  l   ll (x_0, x_1, x_2, x_3, x_4), amelyb  l a rate r  sz 64 bites, a capacity r  sz pedig 256 bites. A capacity m  rete hat  rozza meg a biztons  gi szintet: a 256 bites capacity 128 bites biztons  got garant  l.

3.4.2. Az ASCON permut  ci  

Az ASCON k  zponti   p  t  eleme egy p permut  ci  , amelyet a vagy b k  r  n   t alkalmaznak ($a = 12$ az inicializ  l  sn  l   s v  gleges  t  sn  l, $b = 6$ az adatfeldolgoz  s sor  n). Minden k  r h  rom r  tegb  l   p  l fel:

1. **Konstans hozz  ad  s** (round constant addition): egy k  r-specifikus, nyilvánosan ismert konstans XOR-olunk az x_2 sz  hoz.
2. **Helyettes  t   r  teg** (substitution layer): 64 darab p  rhuzamos, 5 bites S-box alkalmaz  sa. Az S-boxot nem t  bl  zatk  nt val  s  tj  k meg, hanem bitszint   logikai m  veletekk  nt (AND, OR, XOR, NOT) – ez az   n. *bitsliced* megval  s  t  s. Ennek k  sz  nhet  en az S-box r  teg *konstans idej  * v  grehajt  st biztos  t, ami a cache-id  z  t  sen alapul   oldalsatorna-t  mad  sokat eredend  en kiz  rja.

3. **Lineáris diffúziós réteg:** minden 64 bites szót önmagának forgatott verzióival XOR-olja. Például x_0 esetében: $x_0 \leftarrow x_0 \oplus (x_0 \ggg 19) \oplus (x_0 \ggg 28)$. A forgatási értékek szavanként eltérőek, és úgy lettek megválasztva, hogy a teljes diffúzió néhány kör alatt beálljon.

3.4.3. Az AEAD működési folyamat

Az ASCON-128 AEAD sémájának teljes működése négy fázisra bontható:

1. **Inicializálás:** A 320 bites állapotot a kezdőérték (IV), a kulcs (K) és a nonce (N) konkatenációjából állítjuk össze. Ezt követően 12 körös permutációt alkalmazunk, majd a kulcsot XOR-oljuk az állapot alsó részébe.
2. **Kapcsolódó adatok feldolgozása:** Az AD-t 64 bites blokkokra bontjuk, és minden blokkot XOR-olunk a rate részbe, majd 6 körös permutációt alkalmazunk. A fázis végén egy domain separation konstanst (egyetlen bit) XOR-olunk az állapotba, jelezve az AD és a nyílt szöveg határát.
3. **Titkosítás:** A nyílt szöveg minden 64 bites blokkjánál a rate részt XOR-oljuk a nyílt szöveg blokkal – az eredmény a titkosított szöveg blokk –, majd a rate részt felülírjuk a titkosított blokkal, és 6 körös permutációt alkalmazunk.
4. **Véglegesítés (finalization):** A kulcsot XOR-oljuk az állapotba, 12 körös permutációt alkalmazunk, majd a kimeneti 128 bitet a kulccsal XOR-olva kapjuk a hitelesítő taget (T).

A visszafejtés során a fogadó ugyanezeket a lépéseket hajtja végre (a titkosítási fázisban az irányt megfordítva), és a végén összehasonlítja a számított taget a kapott taggel. Eltérés esetén az üzenetet *elutasítja* – így garantálva, hogy manipulált vagy sérült adat soha ne kerüljön feldolgozásra.

3.4.4. Miért nyerte meg az ASCON a NIST versenyt?

Az ASCON sikere több, egymást erősítő tényezőre vezethető vissza [9]. Elegáns belső felépítése – egyetlen permutáció, amely mind az AEAD, mind a hash funkcionalitást megalapozza – rendkívül egyszerűvé teszi az implementációt. Kis állapotmérete (320 bit) kedvező a korlátozott eszközök számára. A bitsliced S-box réteg természetes védelmet nyújt az oldalsatorna-támadásokkal szemben. Végül az éveken át tartó nyilvános kriptanalízis során a kutatóközösség nem talált hatékony támadást a teljes változat ellen, ami megerősítette a tervezők biztonsági állításait.

3.5. Az algoritmusok összehasonlító áttekintése

A négy vizsgált algoritmus tudatos válogatás eredménye, amelynek célja, hogy a könnyűsúlyú kriptográfia különböző tervezési filozófiáit egyetlen, koherens keretben lehessen vizsgálni. Az alábbi táblázat összefoglalja a legfontosabb strukturális jellemzőket.

3.1. táblázat. A vizsgált algoritmusok összehasonlító táblázata.

Jellemző	AES-128	SPECK 128/128	LEA-128	ASCON-128
Típus	Blokktitkosító	Blokktitkosító	Blokktitkosító	AEAD + Hash
Struktúra	SPN	ARX-Feistel	ARX-GFN	Sponge
Blokkméret	128 bit	128 bit	128 bit	64 bit (rate)
Kulcsméret	128 bit	128 bit	128 bit	128 bit
Körök száma	10	32	24	6/12
Szóméret	8 bit	64 bit	32 bit	64 bit
S-box	8-bit (256 elem)	–	–	5-bit (bitsliced)
Nemlinearitás	S-box tábla	mod. összeadás	mod. összeadás	S-box (logikai)
Szabvány	FIPS 197	–	ISO 29192-2	NIST LWC

A struktúra-diverzitás biztosítja, hogy az összehasonlítás ne csupán implementációs kérdésekre szorítkozzék. Az SPN (AES) a lookup-tábla-alapú konfúziót képviseli, ahol a nemlinearitás egy gondosan tervezett S-boxból ered. Az ARX-Feistel (SPECK) és az ARX-GFN (LEA) az S-box nélküli, aritmetikai nemlinearításra épülő irányzat két változata, amelyek eltérő szómérettel és Feistel-struktúrával dolgoznak. A szivacs-alapú ASCON pedig egy teljesen más paradigmát képvisel, ahol a titkosítás és hitelesítés egyetlen, integrált konstrukcióban valósul meg.

Közös vonás, hogy mind a négy algoritmus 128 bites kulcsot használ, ami egységes alapot teremt a biztonsági szint összehasonlításához. A blokktitkosítók esetében a 128 bites blokkméret biztosítja, hogy a birthday attack (születésnap-i támadás) küszöbe – 2^{64} feldolgozott blokk – a gyakorlati alkalmazásokban nem jelent korlátot. Az ASCON esetében, bár a rate csupán 64 bites, a 256 bites capacity gondoskodik az egyenértékű védelemről.

A következő fejezetekben ezeknek az algoritmusoknak az implementációját, teljesítményét és biztonsági tulajdonságait vizsgáljuk meg részletesen.

4. fejezet

Implementáció

(A fejezet tartalma az implementáció elkészülte után kerül megírásra.)

5. fejezet

Teljesítménymérések és eredmények

(A fejezet tartalma a mérések elvégzése után kerül megírásra.)

6. fejezet

Biztonsági elemzés és összehasonlító értékelés

A kriptográfiai algoritmusok értékelésekor a teljesítmény és az erőforrásigény önmagában nem elegendő – legalább ilyen lényeges, hogy az adott algoritmus milyen szintű védelmet nyújt a különböző támadási módszerekkel szemben. E fejezet a vizsgált négy algoritmus biztonsági tulajdonságait tekinti át a szakirodalom alapján. A tárgyalás elméleti jellegű: nem valósítunk meg konkrét támadásokat, hanem a publikált kriptoanalízis eredményeket összegezzük és hasonlítjuk össze.

6.1. A kriptoanalízis alapfogalmai

6.1.1. Támadási modellek

A kriptoanalízis területén a támadó képességei szerint négy, egyre erősödő modellt szokás megkülönböztetni [1]. A leggyengébb a *csak titkosított szöveget* (ciphertext-only, COA) ismerő támadó, aki kizárólag a titkosított kimenetet látja. Az *ismert nyílt szöveges* (known-plaintext, KPA) modellben a támadó rendelkezik néhány (plaintext, ciphertext) párral. A *választott nyílt szöveges* (chosen-plaintext, CPA) modellben a támadó szabadon választhatja meg a titkosítandó üzeneteket – ez a modell a gyakorlatban realisabb, mint elsőre tűnik, mivel sok protokollban a támadó képes szándékosan olyan üzeneteket előidézni, amelyeket a céleszköz titkosít. Végül a *választott titkosított szöveges* (chosen-ciphertext, CCA) modellben a támadó a visszafejtendő szövegeket is kiválaszthatja. A modern kriptográfiai tervezés általában a CPA vagy CCA modellben kíván biztonságot garantálni.

6.1.2. Biztonsági szint és biztonsági margó

Egy algoritmus *biztonsági szintje* azt fejezi ki, hogy hány műveletet igényel a legjobb ismert támadás. Ha az algoritmus kulcsmérete k bit, és a legjobb ismert támadás 2^s műveletet igényel, akkor az algoritmus s -bites biztonságot nyújt. Ideális esetben $s = k$ (a legjobb támadás a brute force), ami 128 bites kulcsnál $2^{128} \approx 3,4 \times 10^{38}$ műveletet jelent – ez a jelenlegi technológiai szint mellett a belátható jövőben nem kivitelezhető.

A *biztonsági margó* (security margin) egy másik fontos mérőszám, amely az algoritmus teljes körszáma (N) és a legjobb ismert támadás által feltörhető körök száma (n) közötti különbséggént definiálható: $\text{margó} = N - n$. Minél nagyobb a margó, annál nagyobb az algoritmus „tartáléka” a jövőbeli, esetleg erősebb támadásokkal szemben [7]. Egy nullás vagy negatív margó azt jelezné, hogy az algoritmus a teljes körszámánál is feltörhető – ilyen algoritmust természetesen nem szabad alkalmazni.

6.2. A főbb támadástípusok áttekintése

6.2.1. Differenciális kriptanalízis

A differenciális kriptanalízist Biham és Shamir vezette be 1991-ben, eredetileg a DES elemzésére [19]. Az alapötlet a következő: a támadó két nyílt szöveget választ, amelyek között *ismert különbség* (differencia, ΔP , jellemzően XOR-ban értelmezve) áll fenn, majd megvizsgálja, hogy ez a különbség hogyan terjed a titkosítás körein keresztül. Ha az S-box vagy az ARX struktúra nem tökéletesen „szórja szét” a differenciákat, akkor bizonyos bemeneti különbségek a kimeneten is statisztikailag kimutatható összefüggéseket hagynak – ezekből a körkulcs bitjeire lehet következtetni.

Az algoritmus tervezők ezért különös gondot fordítanak arra, hogy az S-boxok differenciális eloszlása a lehető legegyszerűsebb legyen, és a körszámot úgy választják meg, hogy a differenciális karakterisztikák valószínűsége a teljes titkosítón át elhanyagolhatóan kicsi legyen. Az ARX struktúráknál (SPECK, LEA) a differenciális elemzés bonyolultabb, mert a moduláris összeadás differenciális viselkedése a carry propagation miatt nehezebben jósolható, mint egy S-box tábláé.

6.2.2. Lineáris kriptanalízis

Matsui 1993-ban publikálta a lineáris kriptanalízis módszerét, szintén a DES-re alkalmazva [20]. A módszer lényege, hogy a támadó lineáris összefüggéseket (XOR-okat) keres a bemeneti bitek, a kimeneti bitek és a kulcs bitjei között. Ha egy ilyen lineáris közelítés torzítása (bias) nem nulla – vagyis az összefüggés a véletlennél gyakrabban teljesül –, akkor kellő számú ismert (plaintext, ciphertext) pár birtokában a kulcs bitjei kinyerhetők.

A védekezés hasonló, mint a differenciális esetben: az S-boxok lineáris közelíthetőségének minimalizálása és a körszám megfelelő megválasztása. Bogdanov és Rijmen munkája [16] a nulla korrelációjú lineáris burkolók (linear hulls with correlation zero) fogalmát vezette be, amely finomította a lineáris támadási modelleket.

6.2.3. Algebrai támadások

Az algebrai támadások az algoritmust a $GF(2)$ véges test feletti egyenletrendszerként írják fel, és megkísérik ezt a rendszert megoldani – amennyiben ez sikerül, a kulcs közvetlenül kiszámítható. A megoldhatóság az egyenletrendszer méretétől és fokszámától függ: az S-box-alapú algoritmusok (AES) viszonylag alacsony fokú egyenleteket generálnak, de a rendszer mérete olyan nagy, hogy a jelenlegi algebrai megoldó módszerek (Gröbner-bázis, SAT solver) nem képesek a teljes algoritmust kezelni. Az ARX struktúráknál (SPECK, LEA) a moduláris összeadás bitszintű reprezentációja bonyolultabb egyenleteket eredményez [7].

6.2.4. Rotációs kriptanalízis

A Khovratovich és Nikolić által 2010-ben bevezetett rotációs kriptanalízis [21] kifejezetten az ARX struktúrákra lett kifejlesztve. Az alapötlet az, hogy ha a bemeneti szavakat forgatjuk (rotate), egy „ideális” ARX rendszerben a kimenet is forgatott lenne – a moduláris összeadás carry propagation-ja azonban megtöri ezt az összefüggést. A támadás azt vizsgálja, hogy ez a törés mennyire kiszámítható statisztikailag.

Mind a SPECK, mind a LEA ellen végeztek rotációs kriptanalízist, de a teljes változatok ellen *nem találtak hatékony támadást*. Az eredmény nem meglepő: a tervezők éppen a carry propagation okozta nemlinearitásra építik a biztonságot, és a forgatási paramétereket úgy választják meg, hogy a rotációs tulajdonság a lehető leggyorsabban elbomoljon [11].

6.2.5. Oldalsatorna-támadások

Az oldalsatorna-támadások (side-channel attacks) nem az algoritmust *matematikai értelemben* támadják, hanem annak *fizikai megvalósítását* használják ki [22]. A végrehajtási idő eltérései (timing attack), az áramfelvétel változásai (simple/differential power analysis, SPA/DPA), az elektromágneses kisugárzás mintái és a cache-memória elérési mintázatai (cache-timing attack) mind információt szivárogtathatnak a kulcsról.

Az SPN struktúrák (AES) S-box-keresési táblája klasszikus célpont a cache-timing támadásoknak: a kulcs egyes biteitől függően más-más S-box bejegyzésekhez fordul a

program, és a cache-találatok/-tévesztések mérésével a kulcs kinyerhető. Az Intel AES-NI utasításkészlet éppen ezt küszöböli ki a hardveres megvalósítással.

Az ARX struktúrák (SPECK, LEA) természetüknél fogva kedvezőbb helyzetben vannak: az összeadás, forgatás és XOR műveletek végrehajtási ideje *nem függ a bemenet értékétől* (nincs adat-függő elágazás vagy memóriakeresés), így a szoftveres implementáció is természetesen konstans idejű. Az ASCON bitsliced S-box rétege hasonlóan konstans idejű végrehajtást biztosít, ami a tervezők explicit célja volt [9].

A jelen dolgozatban oldalsatorna-támadásokat nem valósítunk meg és nem mérünk – ehhez oszcilloszkóp, fizikai hozzáférés az eszközhez és speciális mérőberendezés lenne szükséges. A fentiek elméleti áttekintésként szolgálnak, amelyek az algoritmusok összehasonlító értékelésében fontos szempontot képeznek.

6.2.6. Születésnap-i támadás

A születésnap-i támadás (birthday attack) a valószínűségszámítás születésnap-i paradoxonán alapul: n bites blokkméret esetén $2^{n/2}$ blokk feldolgozása után nagy valószínűséggel előfordul két azonos állapot (ütközés), ami biztonsági problémákat okozhat – például az ECB üzemmódban a titkosított blokkok ismétlődése nyilvánvalóvá válik, vagy a CBC-MAC hamisítható.

Mind a négy vizsgált algoritmusunk 128 bites blokkot kezel (az ASCON esetében a 256 bites capacity határozza meg a releváns küszöböt), így a birthday küszöb 2^{64} , ami a legtöbb IoT alkalmazásban nem érhető el. Ez az egyik ok, amiért 128 bites blokkméretű algoritmusokat választottunk a 64 bites változatok helyett.

6.3. Algoritmusonkénti biztonsági profil

6.3.1. AES-128

Az AES-128 a kriptográfia egyik legjobban vizsgált algoritmus. A legjobb ismert differenciális és lineáris támadások az AES-128 10 köréből legfeljebb 7 kört képesek megtörni, ami 3 kör biztonsági margót jelent – százalékosan kifejezve a teljes körszám 30%-a. A biztonsági szint gyakorlatilag eléri a teljes 128 bitet: a legjobb ismert elméleti támadás (a biclique attack) a komplexitást mindössze $2^{126,1}$ -re csökkenti, ami a gyakorlatban továbbra is megvalósíthatatlan [2].

Az AES oldalsatorna-érzékenysége a lookup-tábla-alapú implementációkban nyilvánul meg: a SubBytes lépés S-box-keresése cache-timing támadásokkal feltörhető. Ezt a problémát a hardveres AES-NI utasításkészlet, illetve a bitsliced szoftveres megvalósítás oldja meg – mikrovezérlőkön azonban ezek ritkán állnak rendelkezésre.

6.3.2. SPECK 128/128

A SPECK 128/128 ellen végzett kriptanalízis eredményei szerint a legjobb differenciális támadás 23 kört képes megtörni a teljes 32-ből, míg a legjobb lineáris támadás 22 körig hatol el [11]. Ez 9, illetve 10 kör biztonsági margót jelent, ami az algoritmust a differenciális és lineáris támadásokkal szemben kellően ellenállóvá teszi.

A rotációs kriptanalízis a SPECK ARX struktúrájának vizsgálatára irányult, de a teljes változat ellen nem sikerült a brute force-nál hatékonyabb támadást konstruálni. Az oldalsatorna-tulajdonságok kedvezőek: a konstans idejű ARX műveletek nem szivárogtatnak időzítési információt. A SPECK egyetlen valódi gyengesége nem kriptográfiai, hanem reputációs: az NSA mint tervező intézmény és a hiányzó teljes design rationale a bizalmi megítélést rontja.

6.3.3. LEA-128

A LEA-128 biztonsági profilja a SPECK-éhez hasonló, de néhány lényeges különbséggel. A legjobb ismert differenciális támadás 15 kört tör meg a 24-ből, ami 9 kör biztonsági margónak felel meg. A lineáris támadás 13 körig hatol el, vagyis 11 kör margó áll rendelkezésre [12]. A rotációs kriptanalízis szintén nem hozott áttörést a teljes változat ellen.

A LEA biztonsági megítélését erősíti, hogy a tervezők a design rationale-t teljes mértékben nyilvánosságra hozták, beleértve a forgatási paraméterek választásának indoklását és a differenciális/lineáris tulajdonságok elemzését. Az ISO/IEC 29192-2:2019 szabványba való bekerülés azt jelzi, hogy a nemzetközi szabványügyi szervezet is elfogadhatónak ítélte a LEA biztonsági érvrendszerét [13].

6.3.4. ASCON-128

Az ASCON biztonsági tulajdonságai más keretben értelmezendők, mint a blokktitkosítóké, mivel az ASCON natívan AEAD sémaként működik. A 256 bites capacity biztosítja a 128 bites biztonsági szintet mind a titkosság, mind a hitelesség tekintetében. A legjobb ismert differenciális támadás a permutáció mintegy 7 körét képes megtörni a 12-ből (az inicializálási/véglegesítési permutációnál), ami 5 kör biztonsági margót jelent [9].

Az ASCON különleges erőssége az oldalsatorna-ellenállás: a bitsliced S-box implementáció természeténél fogva konstans idejű, nem tartalmaz memória-kereséseket vagy adat-függő elágazásokat. A nonce misuse (nonce újrafelhasználás) azonban kritikus gyengeség: ha ugyanazt a nonce-ot kétszer használják ugyanazzal a kulccsal, a titkosság *teljes mértékben* sérül – bár a hitelesség megmarad. Ez nem az algoritmus strukturális hibája, hanem egy tervezési kompromisszum, amelyre az implementáció során kiemelt figyelmet kell fordítani.

6.4. Összehasonlító biztonsági értékelés

Az alábbi táblázat összefoglalja az egyes algoritmusok biztonsági tulajdonságait a szakirodalom alapján.

6.1. táblázat. A vizsgált algoritmusok összehasonlító biztonsági profilja.

Szempont	AES-128	SPECK 128/128	LEA-128	ASCON-128
Kulcsméret (bit)	128	128	128	128
Biztonsági szint	~128 bit	128 bit	128 bit	128 bit
Diff. margó (kör)	3/10	9/32	9/24	5/12
Lin. margó (kör)	3/10	10/32	11/24	–
Birthday védelem	✓	✓	✓	✓
Oldalsatorna	közepes*	jó	jó	kiváló
Szabvány	FIPS 197	–	ISO 29192-2	NIST LWC
Design rationale	teljes	hiányos	teljes	teljes

*Az AES oldalsatorna-érzékenysége lookup-tábla-alapú implementációkra vonatkozik; AES-NI vagy bitsliced megvalósítás esetén a védelem kiváló.

A táblázat adataiból több fontos következtetés vonható le. Bár a biztonsági margó abszolút értékben az AES-nél a legkisebb (3 kör), ez százalékosan 30%-nak felel meg – ami a több évtizednyi intenzív kriptóanalízis fényében elfogadhatónak tekinthető. A SPECK és a LEA hasonló abszolút margóval rendelkeznek (9 kör), de a SPECK-nél ez 32 körből, a LEA-nál 24-ből értendő, ami relatíve a LEA-nál szűkebb mozgásteret jelent. Az ASCON margója (5 kör a 12-ből) az inicializálási permutáció kontextusában értelmezendő, ahol a capacity rész biztosítja a további védelmet.

Az oldalsatorna-tulajdonságok terén az ASCON emelkedik ki, bitsliced S-box implementációjának köszönhetően. Az ARX-alapú SPECK és LEA szintén kedvező tulajdonságokkal rendelkeznek, míg az AES lookup-tábla-alapú megvalósítása e téren sebezhetőbb – bár ez hardveres gyorsítással vagy gondos szoftveres implementációval orvosolható.

A szabványosítottság és a design rationale transzparenciája a bizalmi megítélés szempontjából lényeges: a SPECK ebben a dimenzióban hátrányban van a többi vizsgált algoritmussal szemben, ami nem biztonsági, hanem reputációs kockázatot jelent.

7. fejezet

Összefoglalás és továbbfejlesztési lehetőségek

(A fejezet tartalma a dolgozat véglegesítésekor kerül megírásra.)

Irodalomjegyzék

- [1] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed., Pearson, 2017.
- [2] C. Paar and J. Pelzl, *Understanding Cryptography: A Textbook for Students and Practitioners*, Springer, 2010.
- [3] C. E. Shannon, *Communication Theory of Secrecy Systems*, Bell System Technical Journal **28** (1949), no. 4, 656–715.
- [4] NIST, *Advanced Encryption Standard (AES)*, FIPS PUB 197, National Institute of Standards and Technology, 2001.
- [5] J. Daemen and V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*, Springer, 2002.
- [6] K. A. McKay, L. Bassham, M. S. Turan, and N. Mouha, *Report on Lightweight Cryptography*, NIST Internal Report 8114 (2017).
- [7] A. Biryukov and L. Perrin, *State of the Art in Lightweight Symmetric Cryptography*, IACR ePrint Archive 2017/511 (2017).
- [8] G. Hatzivasilis, K. Fysarakis, I. Papaefstathiou, and C. Manifavas, *A Review of Lightweight Block Ciphers*, Journal of Cryptographic Engineering **8** (2018), no. 2, 141–184.
- [9] C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, *Ascon v1.2: Lightweight Authenticated Encryption and Hashing*, Journal of Cryptology **34** (2021), no. 3.
- [10] NIST, *Lightweight Cryptography Standardization Process: NIST Selects Ascon*, National Institute of Standards and Technology, 2023.
- [11] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and L. Wingers, *The SIMON and SPECK Families of Lightweight Block Ciphers*, IACR ePrint Archive 2013/404 (2013).
- [12] D. Hong, J.-K. Lee, D.-C. Kim, D. Kwon, K. H. Ryu, and D.-G. Lee, *LEA: A 128-Bit Block Cipher for Fast Encryption on Common Processors*, WISA 2013, Lecture Notes in Computer Science **8267** (2013), 3–27.
- [13] ISO/IEC, *Information security – Lightweight cryptography – Part 2: Block ciphers (ISO/IEC 29192-2:2019)*, International Organization for Standardization, 2019.
- [14] D. Dinu, A. Biryukov, J. Großschädl, D. Khovratovich, Y. Le Corre, and L. Perrin, *Triathlon of Lightweight Block Ciphers for the Internet of Things*, NIST Lightweight Cryptography Workshop (2015).
- [15] M. Cazorla, K. Marquet, and M. Minier, *Survey and Benchmark of Lightweight Block Ciphers for Wireless Sensor Networks*, SECRIPT 2013 (2013), 543–548.

- [16] A. Bogdanov and V. Rijmen, *Linear Hulls with Correlation Zero and Linear Cryptanalysis of Block Ciphers*, Designs, Codes and Cryptography **70** (2014), no. 3, 369–383.
- [17] M. M. Hossain, M. Fotouhi, and R. Hasan, *Towards an Analysis of Security Issues, Challenges, and Open Problems in the Internet of Things*, IEEE World of Wireless, Mobile and Multimedia Networks (WoWMoM) (2015), 1–6.
- [18] A.-R. Sadeghi, C. Wachsmann, and M. Waidner, *Security and Privacy Challenges in Industrial Internet of Things*, 52nd ACM/EDAC/IEEE Design Automation Conference (DAC) (2015), 1–6.
- [19] E. Biham and A. Shamir, *Differential Cryptanalysis of DES-like Cryptosystems*, Journal of Cryptology **4** (1991), no. 1, 3–72.
- [20] M. Matsui, *Linear Cryptanalysis Method for DES Cipher*, EUROCRYPT '93, Lecture Notes in Computer Science **765** (1993), 386–397.
- [21] D. Khovratovich and I. Nikolić, *Rotational Cryptanalysis of ARX*, FSE 2010, Lecture Notes in Computer Science **6147** (2010), 333–346.
- [22] P. C. Kocher, *Timing Attacks on Implementations of Diffie-Hellman, RSA, DSS, and Other Systems*, CRYPTO '96, Lecture Notes in Computer Science **1109** (1996), 104–113.

Köszönetnyilvánítás

(A köszönetnyilvánítás szövege a dolgozat véglegesítésekor kerül megírásra.)

Nyilatkozat

Alulírott, Kárász Barna Kristóf, mérnökinformatikus BSc szakos hallgató, kijelentem, hogy a szakdolgozatban ismertettek saját munkám eredményei, és minden felhasznált, nem saját munkából származó eredmény esetén hivatkozással jelöltem annak forrását.

Szeged, 2026. június 24.

Kárász Barna Kristóf

Mellékletek

(A mellékletek a dolgozat véglegesítésekor kerülnek elhelyezésre.)